

Practical 7 – Problem sheet

S Exercise 1

This (slightly artificial) example will show you how to create classes and objects, and how to interact with them in the BlueJ interface. We will program a class that encapsulates a rotation angle in the plane.

- (a) Create a new class `PlanarAngle`. To that end, in the BlueJ menu, select “Edit → New class”. Type in the class name, select “Class Type” as “Class” (this may already be selected), and click “OK”.
- (b) Open the source code for the newly created class. This will contain some example code inside the class which BlueJ has added. You may decide to delete this example code, but you may also modify it in the course of this exercise to meet your needs.
- (c) Add a field named `angle` of type `double` to the class. This is supposed to hold the value of the angle in radians, between 0 and 2π .
- (d) Implement a constructor. It should take one argument (the angle in radians, as a `double` number) and it should store the value of the angle into the field defined in the last step. *Hint:* the constructor declaration starts like

```
public PlanarAngle( ... ) {
```
- (e) Compile your code. Then create an object (an instance of your class): Go to the main BlueJ screen, right-click on the class, and select “new PlanarAngle(double)”. This will invoke your constructor. Having completed this, you will see the newly created object visualized as a red box on the bottom-left of the screen. Double-click on the red box to check that the field of the object does indeed contain the value you expected.
- (f) Go back to your source code. Add a method `asRadians()` that returns the value of the angle in radians. (That is, the method just reads the value of the field `angle` and returns it as a `double` number. It takes no arguments.) See Lecture 14 for examples (there with a complex number). Compile your code.
- (g) To test the method, go to the main screen again, and create an instance of the class `PlanarAngle` as in (e). Right-click on the object (the red box) and select “asRadians()”. Check that the output is as expected.
- (h) Now add a method `asDegrees()` to the class, which is similar to `asRadians()` but returns the angle in degrees instead. Test your code as in (g).
- (i) As a final step, run the unit test `PlanarAngleTest` that is provided for you.

I Exercise 2

The Fibonacci numbers are a sequence of integers F_n ($n \geq 0$) that are defined by the recurrence relation

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2). \quad (1)$$

Write a function `fibonacci` that, given an integer (`int`) n , computes the n th Fibonacci number F_n . Since the Fibonacci numbers grow very fast, please use the `BigInteger` class for computing

F_n , so that integer overflow problems are avoided. Your function should return F_n in decimal representation as a `String`. You can assume $n \geq 0$.

Hint: For handling `BigInteger` objects, see the example code for computing factorials from Lecture 14.

I Exercise 3

In this exercise, we add some more methods to the class `ComplexNumber` that was introduced in the lectures. The class is given (as far as implemented in Lecture 14) in the code template. In parts (a)-(c), note that your code needs to go *into the class* `ComplexNumber`, not into `ComplexComputation`. The class `ComplexComputation` is relevant *only* for part (d).

If you can't recall the formulas for algebraic operations with complex numbers, you may want to look them up, e.g., [here](#) or [here](#).

- Within the class `ComplexNumber`, implement methods `negate()`, `conjugate()`, and `invert()` which return $-z$, $\bar{z}$, and $1/z$, respectively. All these methods take no parameters and their return type is `ComplexNumber`. Here and in the following, z always denotes the complex number represented by the “current” object.
- Within the class `ComplexNumber`, implement methods `subtract(...)` and `divide(...)`, which take a complex number (say w) as their only argument and compute $z - w$ and z/w , respectively. Parameter type and return type is `ComplexNumber` in both cases.
- Within the class `ComplexNumber`, implement methods `exp()` and `log()` (with no parameters) that compute $\exp(z)$ and $\log(z)$ respectively, and return the result as a `ComplexNumber` object. To that end, make use of the standard functions in `java.lang.Math.*` (Remember $\log z = \log |z| + i \arg z$.)
- In the class `ComplexComputation`, implement a function `computationD()` (with no parameters) that computes the following expression and returns it as a `double` number:

$$\operatorname{Re} \frac{\sqrt[3]{1 - \frac{i}{2}}}{2 + 3i} \quad (2)$$

Make use of the `ComplexNumber` class and its methods implemented before. Hint: $z^a = \exp(a \log z)$ for any $z \in \mathbb{C}$, $a \in \mathbb{R}$.

In all parts of the exercise, keep in mind that the class `ComplexNumber` is designed to be immutable, that is, the values of the fields of the object should be set only once, when the object is created, and not modified afterwards. Also, throughout the exercise, you can ignore the occurrence of any exceptional values – that is, you don't need to provide code for the case of a division by zero, for the case that arguments of methods are `null`, etc.

A Exercise 4

When dealing with monetary values, one usually wants to compute with decimal fractions (pounds and pence, etc.). However, it is crucial to avoid overflow problems (like they can

*In particular, `Math.exp(...)` and `Math.log(...)`, which compute the exponential and logarithm of a *real* number, will be useful.

occur with standard integers) and roundoff problems (like they can occur with floating point numbers). When rounding occurs, it needs to be done in a well-controlled way.

For these use cases, the Java class libraries contain the class `java.math.BigDecimal`, which is similar to the class `BigInteger` discussed in the lectures. For example, the following code creates an arbitrary-precision decimal fraction that is initially 0 and will, in all arithmetic operations, produce results accurate to 2 decimal digits:

```
BigDecimal x = new BigDecimal("0.00");
```

Review the Java API documentation for this class. Then, revisit the class `BankAccount` as introduced in Lecture 13. We want to modify this class slightly. It should still have fields `balance`, `accountNumber` and `overdraftLimit`, a constructor `BankAccount(int number)`, and methods `deposit(...)`, `withdraw(...)` and `currentBalance(...)`. However, we want to make the following changes:

- The account balance should now be held as a `BigDecimal`, accurate to 2 digits after the decimal point. The parameter types and the return types of the methods need to be adjusted accordingly.
- The method `withdraw` should still check whether enough funds are available for the withdrawal. If withdrawal would result in a balance lower than the negative `overdraftLimit`, then no money should be withdrawn. The method should return a `boolean` that indicates whether the withdrawal was successful. Note that both `overdraftLimit` and the parameter of `withdraw(...)` are now of `BigDecimal` type. The overdraft limit should initially be set to 100.00.
- The method `currentBalance()` should return the current account balance as a `BigDecimal`. In addition, implement a method `currentBalanceAsString()` which returns the current account balance as a character string in decimal representation.

Start by creating a new class `BankAccount`. It may be useful to copy some code from Lecture 13 in and modify it.